Highlights

Preference-Safe Minimum-Coverage Top-N Recommendation via Minimum-Cost Flow

Author A, Author B

- A preference filter prevents coverage gains from relying on low-ranked items.
- The model is equivalent to minimum-cost flow with one lower-bounded arc.
- TPCAR parameterizes exact computation by the realized coverage gap.
- BGCR delivers near-optimal solutions in seconds on very large candidate graphs.

Preference-Safe Minimum-Coverage Top-N Recommendation via Minimum-Cost Flow

Author A[a,*], Author B[b]

[a]Author's Affiliation, Address, City, Postcode, Country
[b]Author's Affiliation, Address, City, Postcode, Country

ARTICLE INFO

Keywords:
Top-N recommendation
aggregate diversity
minimum coverage requirement
minimum-cost flow
exact algorithm

ABSTRACT

We study minimum-coverage Top-N recommendation with preference-based candidate filtering. Given predicted user–item scores, the filter retains a high-ranked candidate set for each user, after which the platform maximizes total predicted utility subject to a lower bound on the number of distinct items appearing across all recommendation lists. We prove that the resulting binary program is equivalent to a minimum-cost flow problem with a single lower-bounded arc. This structure yields two complementary algorithms. Two-Phase Coverage Augmentation for Recommendation (TPCAR) starts from the unconstrained optimum and performs implicit shortest-path augmentations only for the realized coverage gap. It returns a globally optimal solution whenever the target is feasible and identifies the maximum attainable coverage when it is not. Batched Gap-aware Coverage Reallocation (BGCR) applies low-loss reallocations in batches to construct high-quality solutions under stringent time or memory budgets. Experiments on five real-world datasets show gains in aggregate diversity of 29.49%–757.80%, while the largest relative decrease in average predicted utility is approximately 0.63%. On instances with independent exact benchmarks, TPCAR matches the benchmark objective values and is more than three orders of magnitude faster than a general-purpose integer programming solver in the low-coverage OP regime. BGCR reduces several hour-scale exact runs to seconds while incurring only small losses in predicted utility on the reported instances. The results show that a minimum coverage requirement, coupled with preference filtering, provides an interpretable and computationally tractable mechanism for managing catalog exposure.

1. Introduction

Digital platforms routinely select a small number of items from catalogs that may contain millions of alternatives. For each user, the platform constructs a personalized list of length N using relevance scores estimated from past interactions (Gao, Zheng, Li et al., 2023; Koren, 2010). The resulting lists should provide high user-level utility while distributing recommendation opportunities across a sufficiently broad set of items. Maximizing predicted utility alone can concentrate exposure on a narrow group of popular items, reinforce popularity bias, and reduce aggregate diversity (Adomavicius and Kwon, 2012, 2014; Muter and Aytekin, 2017).

The prediction layer of a recommender system estimates a user's preference for items not previously consumed. Representative approaches include item-based collaborative filtering (Sarwar, Karypis, Konstan and Riedl, 2001), neighborhood models and matrix factorization (Koren, 2008; Koren, Bell and Volinsky, 2009; Koren, 2010), Bayesian personalized ranking (Rendle, Freudenthaler, Gantner and Schmidt-Thieme, 2009), and neural collaborative filtering (He, Liao, Zhang, Nie, Hu and Chua, 2017). These techniques are used in large-scale industrial systems (Covington, Adams and Sargin, 2016), and recent work has extended preference modeling to graph neural networks, large language models, and review text (Gao et al., 2023; Wu, Zheng, Qiu et al., 2024; Son, Kim and Kim, 2025). Once predicted scores are available, conventional Top-N recommendation selects the N highest-scoring items independently for each user. This rule maximizes each user's predicted utility but does not control the number of distinct items exposed across users (Abdollahpouri, Burke and Mobasher, 2017; Mansoury, Abdollahpouri, Pechenizkiy, Mobasher and Burke, 2020).

Optimization-based work addresses this system-level concern explicitly. Adomavicius and Kwon (2014) maximize recommendation accuracy subject to a prescribed level of aggregate diversity. Muter and Aytekin (2017) incorporate distributional diversity and develop scalable solution procedures, while Canakoglu, Muter and Aytekin (2021) integrate accuracy, distributional diversity, and individual diversity in a multiobjective Top-N framework. These studies establish the main optimization paradigm for coordinating recommendation utility and diversity. Two issues nevertheless remain. First, an aggregate-diversity constraint may induce the optimizer to select items that rank poorly for particular users. Such mismatches can erode trust and impose longer-term costs on the platform (De Biasio, Navarin and Jannach, 2024). Second, exact implementations of aggregate-diversity-constrained Top-N models rely primarily on general-purpose optimization software and do not fully exploit their combinatorial structure at the scale of modern candidate graphs.

We address both issues by adding preference-based candidate filtering to an aggregate-diversity-constrained Top-N model and exposing its minimum-cost flow structure.

*Corresponding author.
author1@university.edu (A. A); author2@university.edu (A. B)
ORCID(s):

The filter restricts each user's eligible items to a high-ranked, preference-supported candidate set. On this filtered graph, the platform maximizes total predicted utility while requiring the union of all recommendation lists to contain at least D distinct items. We exploit the resulting single lower-bounded coverage arc and the separability of the unconstrained solution to develop an exact coverage-augmentation method for large instances and a fast heuristic for larger, resource-constrained instances.

The paper makes four contributions.

(1) We extend an aggregate-diversity-constrained Top-N formulation with a preference-based candidate filter. The parameter D specifies the platform's system-wide coverage floor, whereas the retention rate α restricts assignments to high-ranked user-specific candidates. This separation allows the platform to increase aggregate diversity without assigning items outside the retained high-preference sets.

(2) We construct a network with a single lower-bounded coverage arc and prove a one-to-one correspondence between feasible integer flows and feasible recommendation assignments. The equivalence turns exact solution of the binary model into a polynomial-time minimum-cost flow problem and provides an explicit structural explanation for integrality.

(3) We develop two complementary algorithms. TPCAR starts from the decomposable unconstrained optimum and performs implicit shortest-path augmentations. Its augmentation count is driven by the realized coverage gap $\max\{D - K^0, 0\}$ rather than the total recommendation flow $N|U|$; it preserves global optimality and identifies the maximum attainable coverage when the nominal target is infeasible. BGCR replaces global augmenting paths with batched low-loss reallocations and is designed for candidate graphs that are too large, or time budgets that are too tight, for repeated exact shortest-path computations.

(4) We provide a systematic computational study on five real-world datasets. The experiments quantify the utility–coverage trade-off, validate exact objective values against independent benchmarks, isolate the effect of preference filtering at identical coverage requirements, and delineate the operating regions of TPCAR, explicit-network minimum-cost flow (MCF), and BGCR. TPCAR is most attractive when the coverage gap is small; explicit-network MCF can be preferable when the gap is large and the graph fits in memory; BGCR is appropriate for very large graphs or stringent time budgets.

Figure 1 summarizes the decision setting and solution architecture. Its center depicts how the preference-filtered user–item graph feeds a coverage aggregation node in the minimum-cost flow network. Section 2 reviews related work. Section 3 formulates the problem and establishes the flow equivalence. Section 4 presents TPCAR and BGCR. Section 5 reports the computational study, and Section 6 concludes.

2. Related literature

Our work lies at the intersection of preference estimation, aggregate diversity optimization, and large-scale network optimization. Preference-estimation methods derive user–item scores from historical interactions. Item-based collaborative filtering (Sarwar et al., 2001), neighborhood and matrix-factorization models (Koren, 2008; Koren et al., 2009; Koren, 2010), Bayesian personalized ranking (Rendle et al., 2009), and neural collaborative filtering (He et al., 2017) provide widely used foundations, including for industrial recommendation pipelines (Covington et al., 2016). More recent research uses graph neural networks and large language models to enrich representations (Gao et al., 2023; Wu et al., 2024). For review-based prediction, Son et al. (2025) propose LETTER, which combines rating-aware homogeneous review graphs with an explicit distinction between user likes and dislikes. We use LETTER to generate the predicted scores in our experiments. Whereas these methods ask which items a user is likely to prefer, our problem begins after scores have been estimated and asks how limited recommendation positions should be allocated across users.

Aggregate diversity measures the number of distinct items appearing in the union of all users' recommendation lists (Adomavicius and Kwon, 2012; Muter and Aytekin, 2017). Adomavicius and Kwon (2012) show that ranking rules can substantially broaden this union with modest losses in accuracy. Adomavicius and Kwon (2014) subsequently formulate several optimization models for maximizing aggregate diversity and coordinating it with accuracy. Muter and Aytekin (2017) study scalable formulations that also capture how evenly recommendations are distributed across items, and Canakoglu et al. (2021) jointly consider accuracy, aggregate diversity, and individual diversity. Together, these papers establish aggregate diversity optimization as a cross-user allocation problem. Aggregate diversity gains, however, should not come from assigning users items with little preference support. The systematic review of De Biasio et al. (2024) emphasizes that poor recommendations may reduce trust, contribute to churn, and impose economic costs.

The solution methods in this stream include greedy rules, network-flow models, and integer programming (Adomavicius and Kwon, 2014). Greedy methods are computationally attractive but generally lack global optimality guarantees. Existing network formulations are effective when aggregate diversity is the primary objective, but they do not directly expose how to enforce a platform-specified diversity floor while maximizing recommendation utility. Integer programming represents this trade-off exactly, yet generic solvers become costly as the user–item graph grows. Muter and Aytekin (2017) establish total unimodularity for a related

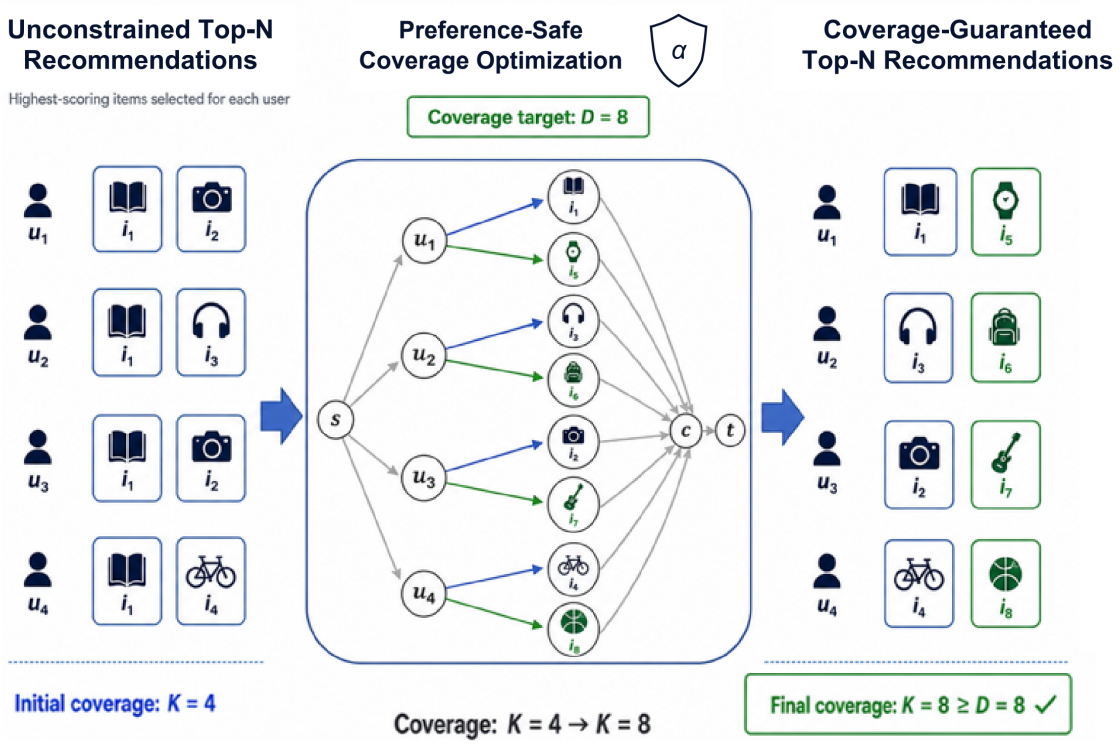

Figure 1: Preference-safe minimum-coverage Top-N recommendation framework.

formulation, but the literature has not developed a specialized exact algorithm parameterized by the coverage gap for this aggregate-diversity-constrained model.

We fill this gap at three levels. At the modeling level, D specifies a lower bound on aggregate diversity, and α restricts coverage adjustments to a preference-supported candidate graph. At the structural level, the model becomes a minimum-cost flow problem in which all cross-user coupling is concentrated in one lower-bounded arc. At the algorithmic level, the unconstrained solution decomposes by user, enabling TPCAR to repair only the realized coverage gap; BGCR provides a faster alternative when exact repair is too costly. The same combination of limited assignment capacity, eligibility restrictions, and coverage requirements also arises in advertising, content distribution, and information delivery.

3. Preference-safe minimum-coverage Top-N recommendation

3.1. Problem definition and integer programming formulation

We consider a single recommendation cycle. Let U and I be finite, disjoint index sets of users and items, respectively, and let R^*_{ui} denote the predicted score of item i for user u. The platform must construct a list of N items for every user and seeks to maximize the total predicted score while meeting a minimum item-coverage requirement.

Let $\bar{I}_u$ denote the set of items eligible for user u before filtering. In practice, $|\bar{I}_u|$ is typically much larger than N. To prevent the coverage requirement from inducing low-relevance assignments, we apply a preference-based candidate filter. Given a retention rate $\alpha \in (0, 1]$, sort the items in $\bar{I}_u$ in nonincreasing order of R^*_{ui} and retain the first $\lceil \alpha |\bar{I}_u| \rceil$ items. Denote the resulting set by I^α_u. Smaller values of α impose a stricter eligibility threshold. We assume $|I^\alpha_u| \geq N$ for all $u \in U$. Define the reachable item set $I^\alpha = \bigcup_{u \in U} I^\alpha_u$, the candidate edge set $E^\alpha = \{(u, i) : u \in U,\ i \in I^\alpha_u\}$, and the eligible-user set $U^\alpha_i = \{u \in U : i \in I^\alpha_u\}$ for each item i.

For an integer coverage floor D, binary variable x_{ui} equals one if item i is recommended to user u. Binary variable z_i indicates whether item i is counted toward coverage; whenever $z_i = 1$, the item must be recommended at least once. The preference-safe minimum-coverage model is

$$\text{(P1)} \quad \max \quad \sum_{u \in U} \sum_{i \in I^\alpha_u} R^*_{ui} x_{ui}$$

$$\text{s.t.} \quad \sum_{i \in I^\alpha_u} x_{ui} = N, \qquad \forall u \in U, \tag{1}$$

$$\sum_{u \in U^\alpha_i} x_{ui} - z_i \geq 0, \quad \forall i \in I^\alpha, \tag{2}$$

$$\sum_{i \in I^\alpha} z_i \geq D, \tag{3}$$

$$x_{ui} \in \{0,1\}, \qquad \forall (u,i) \in E^\alpha, \tag{4}$$

$$z_i \in \{0,1\}, \qquad \forall i \in I^\alpha. \tag{5}$$

The predicted scores R^*_{ui}, list length N, and coverage floor D are all fixed before (P1) is solved. Constraints (1) assign exactly N items to each user. Constraints (2) ensure that every item counted toward coverage is assigned at least once, and constraint (3) requires at least D distinct covered items. Constraints (4)–(5) impose binary domains.

Model (P1) extends the aggregate-diversity-constrained formulation of Adomavicius and Kwon (2014) by restricting decisions to the filtered edge set E^α. When $\alpha = 1$, $I^\alpha_u = \bar{I}_u$ and (P1) reduces to the unfiltered model. Decreasing α contracts each user's candidate set and strengthens preference protection. It can also change the graph's coverage capacity. Although every item in I^α is eligible for at least one user, items compete for a finite number of recommendation positions and need not be simultaneously coverable.

Let $K_{\max}(\alpha)$ be the maximum number of distinct items that can be covered on E^α while assigning exactly N items to every user. Model (P1) is feasible if and only if $D \leq K_{\max}(\alpha)$. In general, $K_{\max}(\alpha)$ cannot be inferred from $|I^\alpha|$ alone. A target based only on the number of reachable items may therefore satisfy $D > K_{\max}(\alpha)$. A generic integer programming solver or a fixed-lower-bound flow model will then report infeasibility without identifying the attainable boundary or a recommendation plan at that boundary. TP-CAR, introduced in Section 4.1, instead augments coverage from the unconstrained optimum. It returns a global optimum if $D \leq K_{\max}(\alpha)$ and identifies both $K_{\max}(\alpha)$ and a corresponding recommendation plan otherwise.

3.2. Equivalent minimum-cost flow formulation

Even when D is feasible, solving (P1) on a large graph can be demanding. A generic mathematical programming implementation creates a variable for every candidate edge and maintains the associated coefficient matrix, indices, and solver state. With tens or hundreds of millions of candidate edges, model construction and memory consumption become bottlenecks. We next exploit the combinatorial structure of (P1) to obtain an equivalent minimum-cost flow model.

The minimum coverage requirement is the only coupling among user-level assignment decisions. We separate the first unit of flow through each covered item from any additional assignments of that item and aggregate all first units through one lower-bounded arc. Across all users, there are $N|U|$ recommendation positions. Introduce a coverage aggregation node c and a sink t, and construct a directed network $G = (V, A)$ with

$$V = U \cup I^\alpha \cup \{c, t\}$$

and

$$A = \{(u,i) : (u,i) \in E^\alpha\} \cup \{(i,c) : i \in I^\alpha\} \cup \{(i,t) : i \in I^\alpha\} \cup \{(c,t)\}.$$

Under the convention that net supply equals outflow minus inflow, each user node has supply N, the sink has supply $-N|U|$, and all item nodes and node c have zero supply. Arc bounds, costs, and interpretations are given in Table 1.

One unit on a user–item arc represents one recommendation. For each item i, at most one unit can pass through (i,c) and count toward coverage; all remaining units go directly to t through (i,t). The lower bound on (c,t) forces at least D first units through c. Because user assignments are encoded as node supplies, (c,t) is the only arc with a positive lower bound.

Theorem 1. *Model (P1) is equivalent to the minimum-cost flow model above. In particular, feasible solutions (x,z) of (P1) are in one-to-one correspondence with feasible integer flows in G. If a solution of (P1) has objective value F, the corresponding flow has total cost $-F$.*

Proof. First construct a flow from a feasible solution (x,z) of (P1). Set

$$f_{ui} = x_{ui}, \qquad f_{ic} = z_i,$$
$$f_{it} = \sum_{u \in U^\alpha_i} x_{ui} - z_i, \qquad f_{ct} = \sum_{i \in I^\alpha} z_i.$$

Constraint (1) gives net outflow N at each user node; (2) gives $f_{it} \geq 0$; and (3) gives $f_{ct} \geq D$. The binary domains imply the unit-capacity restrictions on (u,i) and (i,c). Moreover, $f_{it} \leq |U^\alpha_i|$ and $f_{ct} \leq |I^\alpha|$. These definitions satisfy flow balance at every item node and at c, while t receives $N|U|$ units. Hence they define a feasible integer flow.

Conversely, take any feasible integer flow f in G and set $x_{ui} = f_{ui}$ and $z_i = f_{ic}$. The unit capacities on (u,i) and (i,c) imply $x_{ui}, z_i \in \{0,1\}$. User-node supplies yield $\sum_{i \in I^\alpha_u} x_{ui} = N$. Flow balance at item i gives $\sum_{u \in U^\alpha_i} x_{ui} - z_i = f_{it} \geq 0$, and flow balance at c, together with the lower bound on (c,t), gives $\sum_i z_i = f_{ct} \geq D$. Thus (x,z) satisfies all constraints of (P1). The two constructions are inverses. Only user–item arcs have nonzero costs, so total flow cost is $-\sum_{u,i} R^*_{ui} x_{ui}$.

The same two mappings apply to continuous variables. Consequently, the linear programming relaxation of (P1) is the projection of the corresponding network-flow polytope onto the (x,z) space. The node–arc incidence matrix is totally unimodular, and all supplies, capacities, and lower bounds are integral; hence the flow polytope is integral and the relaxation of (P1) has an integer optimum (Schrijver, 1986; Ahuja, Magnanti and Orlin, 1993).

Theorem 1 and the standard optimality conditions for successive shortest paths imply that a specialized minimum-cost flow algorithm solves (P1) globally. The network contains $|U| + |I^\alpha| + 2$ nodes and $|E^\alpha| + 2|I^\alpha| + 1$ arcs. After eliminating the lower bound and converting node supplies in the standard way, node potentials maintain nonnegative reduced costs and each shortest path can be computed with binary-heap Dijkstra. Integrality ensures that each effective augmentation sends at least one unit, so at most $N|U|$ augmentations are required. The resulting time bound is

$$O\big(N|U|\big(|E^\alpha| + |U| + |I^\alpha|\big) \log(|U| + |I^\alpha|)\big),$$

Table 1
Arc classes in the minimum-cost flow network.

Arc	Lower bound	Capacity	Unit cost and interpretation
(u, i)	0	1	$-R^*_{ui}$; item i is assigned to user u
(i, c)	0	1	0; item i is counted toward coverage
(i, t)	0	$\lvert U^\alpha_i \rvert$	0; remaining assignments of item i
(c, t)	D	$\lvert I^\alpha \rvert$	0; minimum coverage requirement

with space $O(|E^\alpha| + |U| + |I^\alpha|)$. Because $N \le |I^\alpha_u| \le |I^\alpha|$, these bounds are polynomial in the explicit network size (Ahuja et al., 1993; Bertsekas, 1998).

4. Algorithms and theoretical analysis

Section 3.2 establishes (P1) as a minimum-cost flow problem with one lower-bounded arc, providing the basis for a specialized exact algorithm. Solving the full network directly nevertheless has two limitations. If D exceeds the attainable coverage limit of the candidate graph, a fixed-lower-bound model reports infeasibility but does not return the maximum attainable coverage together with a plan that attains it. In addition, a standard successive-shortest-path implementation stores the complete residual network and may perform up to $N|U|$ unit augmentations, one for each unit of recommendation flow. This procedure does not exploit the special structure of instances with an unknown coverage limit or a small realized coverage gap.

The key observation is that setting the lower bound of (c, t) to zero makes the problem separable by user: each user independently receives the N highest-scoring eligible items. This gives the unconstrained optimum and its natural coverage K^0 without sending $N|U|$ units through a general flow routine. The additional work induced by the coverage constraint is therefore only to repair the gap $\Delta = D - K^0$ when $D > K^0$. We use this parameterization to develop two complementary methods. TPCAR generates shortest coverage-augmenting paths from the full residual network only as needed and retains global optimality whenever the target is feasible. BGCR restricts the search to single-user reallocations that immediately activate an uncovered item; it sacrifices global optimality and target-attainment guarantees for speed.

4.1. Two-Phase Coverage Augmentation for Recommendation (TPCAR)

TPCAR has two phases. Phase 1 drops the minimum coverage constraint and independently selects the N highest-scoring candidates for every user, yielding an unconstrained optimum x^0. Let

$$r^0_i = \sum_{u \in U^\alpha_i} x^0_{ui},$$

$$C^0 = \{i \in I^\alpha : r^0_i \ge 1\}, \qquad K^0 = |C^0|.$$

We call K^0 the natural coverage. If $K^0 \ge D$, then x^0 is already globally optimal for (P1). Otherwise, TPCAR enters the coverage-augmentation phase.

When $K^0 < D$, represent the current recommendation plan as an integer flow in the network of Section 3.2: exactly one unit associated with each covered item passes through (i, c), and all other units assigned to that item pass through (i, t). In the residual network, every unselected candidate edge creates an arc $u \to j$ of cost $-R^*_{uj}$, whereas every selected edge creates a reverse arc $i \to u$ of cost R^*_{ui}. Residual item and coverage arcs have zero cost. Increasing the flow on (c, t) from k to $k + 1$ is equivalent to finding a residual path from t to c that does not use (c, t) and then closing an augmenting cycle with the forward arc (c, t).

TPCAR generates residual adjacency from the current recommendation state instead of explicitly materializing the network. A typical path is

$$t \to i_0 \to u_1 \to i_1 \to \cdots \to u_p \to i_p \to c.$$

Each segment $i_{\ell-1} \to u_\ell \to i_\ell$ replaces item $i_{\ell-1}$ by item i_ℓ in user u_ℓ's list and has cost $R^*_{u_\ell i_{\ell-1}} - R^*_{u_\ell i_\ell}$. The path begins with an item that has a duplicate assignment still flowing directly to t and ends at an item not yet counted toward coverage. It preserves the list length of every user and all node balances while increasing coverage by exactly one. Triples (u, i, j) record adjacent reverse and forward candidate arcs; they do not require explicit enumeration of exchange combinations.

Lemma 2. *For any integer residual network induced by the current recommendation plan, every simple t–c path in the implicit TPCAR graph corresponds to a residual path in the full network that avoids (c, t). Conversely, every simple augmenting cycle that increases flow on (c, t) by one projects, after removing (c, t), to a t–c path in the implicit graph. The correspondence preserves path cost, keeps every list length equal to N, and increases aggregate coverage by one.*

PROOF. The first arc $t \to i_0$ of an implicit path corresponds to residual arc (t, i_0), which exists exactly when item i_0 has a duplicate assigned unit that is not counted toward coverage. Each intermediate segment $i_{\ell-1} \to u_\ell \to i_\ell$ consists of the reverse residual arc of a selected user–item edge and the forward residual arc of an unselected candidate edge. The last arc $i_p \to c$ is the forward residual arc of an item not currently counted toward coverage. Item and coverage arcs have zero cost, so each exchange segment and the corresponding residual arcs have the same total cost. Augmenting one unit along this path and (c, t) performs the recorded exchanges and counts one new item, preserving user balances and increasing coverage by one.

Algorithm 1 Two-Phase Coverage Augmentation for Recommendation (TPCAR)

Input: $\{I_u^\alpha\}$, $\{R_{ui}^*\}$, list length N, target D
Output: An optimal plan at target or maximum attainable coverage
Set each L_u to its Top-N candidates and initialize x
Compute r_i, $C \leftarrow \{i : r_i > 0\}$, and $K \leftarrow |C|$
if $K \geq D$ **then**
    return x
end if
Initialize the implicit residual graph and potentials π
while $K < D$ **do**
    Find a shortest t–c path P under reduced costs c^π
    if P does not exist **then**
        return the current solution
    end if
    Apply all exchanges on P and update x, r, C, and K
    Update the objective value and potentials π
end while
return x

Conversely, aside from (c,t), a user node is incident only to user–item arcs; an item node is incident only to user–item arcs, (i,c), and (i,t); and c is incident only to item arcs and (c,t). Removing (c,t) from an augmenting cycle and eliminating canceling cycles therefore leaves a residual path alternating among duplicate items, users, and candidate items and ending at an uncounted item. This is precisely a path in the implicit graph.

For an arc (a,b), define reduced cost $c_{ab}^\pi = c_{ab} + \pi(a) - \pi(b)$. Let $\theta_u = \min_{i:x_{ui}^0=1} R_{ui}^*$, initialize $\pi(u) = \theta_u$, and set all other potentials to zero. The Top-N property gives $R_{ui}^* \geq \theta_u \geq R_{uj}^*$ for selected i and unselected j. Reduced costs of selected reverse arcs and unselected forward arcs are therefore $R_{ui}^* - \theta_u \geq 0$ and $-R_{uj}^* + \theta_u \geq 0$, respectively. Standard potential updates after each shortest path preserve nonnegative reduced costs, so Dijkstra's algorithm applies.

Lemma 3. *If Phase 1 yields $K^0 \geq D$, then x^0 is a globally optimal solution of (P1).*

Proof. Solution x^0 maximizes total utility after the coverage constraint is removed. Model (P1) only restricts that feasible region. If x^0 itself meets the restriction, no feasible solution of (P1) can have a larger objective value.

When $K^0 < D$, correctness follows from two invariants: implicit paths are cost-preserving representations of coverage augmentations in the full residual network, and the current flow remains a minimum-cost flow for every attained coverage level.

Theorem 4. *TPCAR terminates after finitely many steps. If $D \leq K_{\max}(\alpha)$, it returns a globally optimal solution of (P1). If $D > K_{\max}(\alpha)$, it identifies $K_{\max}(\alpha)$ and returns an optimal recommendation plan among those attaining that coverage.*

Proof. By Theorem 1, (P1) is an integer minimum-cost flow problem. Let $\mathcal{F}_k$ denote flows in the network of Section 3.2 for which (c,t) carries at least k units. Phase 1 independently selects the Top-N items of every user and therefore produces a minimum-cost flow in $\mathcal{F}_0$. If its natural coverage is K^0, the same flow belongs to every $\mathcal{F}_k$, $0 \leq k \leq K^0$, and remains minimum cost. Thus, when $K^0 < D$, augmentation begins from an optimal integer flow in $\mathcal{F}_{K^0}$.

Suppose the current flow is minimum cost in $\mathcal{F}_k$ and has actual coverage k. If the residual network contains a t–c path that avoids (c,t), augmenting along this path and (c,t) increases coverage by one. By Lemma 2, paths in the implicit and full residual networks have identical costs. Selecting a shortest path therefore minimizes the cost increase among all unit augmentations from coverage k to $k+1$. The standard successive-shortest-path optimality condition (Goldberg and Tarjan, 1990; Orlin, 1997) implies that the augmented flow is minimum cost in $\mathcal{F}_{k+1}$. Induction establishes optimality at every attained coverage level.

Each successful augmentation increases coverage by one, which is bounded by $|I^\alpha|$, so the algorithm terminates. If it stops at D, its flow is minimum cost in $\mathcal{F}_D$ and corresponds to a global optimum of (P1). If no t–c path exists at some $k < D$, no feasible augmentation can further increase coverage. Otherwise, the difference between the current optimal flow and a higher-coverage flow would contain a residual augmentation that increases flow on (c,t), a contradiction. Hence $k = K_{\max}(\alpha)$, and the current plan is optimal at that boundary.

Proposition 5. *TPCAR has time complexity*

$$O\Big(|E^\alpha| \log |I| + \max\{D - K^0, 0\} \cdot \big(|E^\alpha| + |I|\big) \log(|U| + |I|)\Big).$$

If $K^0 \geq D$, only Phase 1 is executed and the bound reduces to $O(|E^\alpha| \log |I|)$.

Proof. Sorting each user's candidates and retaining the first N items takes at most $O(|E^\alpha| \log |I|)$ in total. In any augmentation, each candidate edge generates at most one forward or reverse residual arc. The number of selected reverse arcs is $N|U| \leq |E^\alpha|$, and item-related auxiliary arcs contribute $O(|I|)$. Thus the implicit graph has $O(|E^\alpha| + |I|)$ arcs and $O(|U| + |I|)$ nodes. Binary-heap Dijkstra takes $O((|E^\alpha| + |I|) \log(|U| + |I|))$ per augmentation. Each successful augmentation adds exactly one covered item. A feasible target requires $D - K^0$ augmentations; an infeasible target terminates at $K_{\max}(\alpha)$ and therefore requires no more than $\max\{D - K^0, 0\}$ augmentations.

TPCAR and explicit-network MCF produce the same optimum on feasible instances and both may need to inspect all candidate edges in the worst case. Their computational drivers differ, however. TPCAR reuses the separable unconstrained optimum and reduces the number of exact augmentations from at most $N|U|$ to the coverage gap $\Delta = \max\{D - K^0, 0\}$. It is therefore particularly useful when Δ is small, when the attainable coverage limit is unknown, or when a maximum-coverage solution is required for an infeasible nominal target. A highly optimized explicit-network MCF implementation may be faster when Δ is large and the network fits in memory.

Algorithm 2 Batched Gap-aware Coverage Reallocation (BGCR)

Input: $\{I_u^\alpha\}$, $\{R_{ui}^*\}$, list length N, target D
Output: Plan x, coverage K, objective F, feasibility flag

```
Set each L_u to its Top-N candidates; initialize x, r, C, K, F
while K < D do
    Generate Q = {(u, i, j, δ) : j ∈ I_u^α \ C, i ∈ L_u, r_i ≥ 2}
    Set δ ← R*_ui − R*_uj for each move in Q
    if Q = ∅ then
        break
    end if
    Sort Q by nondecreasing δ; set changed ← 0
    for all (u, i, j, δ) in Q in sorted order do
        if the move remains feasible then
            Replace i by j and update x, r, C, K, F
            changed ← 1; break if K ≥ D
        end if
    end for
    if changed = 0 then
        break
    end if
end while
return x, K, F, 1{K ≥ D}
```

4.2. Batched Gap-aware Coverage Reallocation (BGCR)

Repeated global shortest-path searches can be costly when the coverage gap is large. BGCR uses the same candidate graph and user-level Top-N initialization as TPCAR, but restricts the search to one-step reallocations that directly activate an uncovered item. It sorts these moves by utility loss and accepts currently feasible moves in batches. This restriction preserves every list length and makes coverage nondecreasing, but it may miss augmenting paths that require intermediate exchanges that temporarily leave coverage unchanged. BGCR therefore guarantees neither attainment of D nor global optimality.

Initialization takes $O(|E^\alpha| \log |I|)$ time. A round generates at most $N|E^\alpha|$ candidate reallocations, whose enumeration, sorting, and scan take $O(N|E^\alpha| \log(N|E^\alpha|))$. Every effective round covers at least one new item, so the number of rounds is at most $\max\{D - K^0, 0\}$. The overall bound is

$$O\big(|E^\alpha| \log |I| + \max\{D - K^0, 0\} N|E^\alpha| \log(N|E^\alpha|)\big) .$$

Explicitly storing Q uses $O(N|E^\alpha|)$ space. Blocked generation and external sorting can reduce peak memory without changing this time bound.

TPCAR and BGCR thus share the same initialization and candidate space but provide different guarantees. TPCAR is globally optimal for feasible targets and identifies the attainable boundary otherwise. BGCR returns a rapid coverage-enhanced plan together with its actual coverage and feasibility status. The computational study evaluates these guarantees separately from speed and objective quality.

Table 2
Statistics of the experimental datasets.

Dataset	Users	Items	Reviews	Density
OP	4,905	2,420	53,228	4.49×10^{-3}
Yelp	9,464	11,197	171,003	1.61×10^{-3}
VG	24,303	10,672	231,780	8.90×10^{-4}
TG	19,412	11,924	167,597	7.20×10^{-4}
SO	35,598	18,357	296,337	4.50×10^{-4}

Note: Amazon statistics are from the 5-core datasets (McAuley et al., 2015). Density is the fraction of observed entries in the user–item matrix.

5. Computational experiments

The experiments address four connected questions. Can a minimum coverage requirement substantially increase aggregate diversity at a limited utility cost? Can TPCAR preserve exactness while extending beyond the practical scale of generic integer programming? How much solution quality does BGCR sacrifice in exchange for speed? Finally, how do the nominal coverage ratio $D/|I^\alpha|$ and retention rate α jointly determine attainable coverage, preference protection, and graph size? Except for the filtering control experiment, all algorithms use identical candidate sets, predicted scores, and list lengths.

5.1. Experimental design

Data and predicted scores. We use Office Products (OP), Video Games (VG), Toys and Games (TG), and Sports and Outdoors (SO) from the Amazon 5-core data (McAuley, Targett, Shi and van den Hengel, 2015), together with the public Yelp dataset. Predicted user–item scores R_{ui}^* are generated by LETTER (Son et al., 2025). In the original study, LETTER outperforms nine representative baselines on MAE, MSE, and RMSE across six datasets; on the five datasets used here, its MSE improvement ranges from 1.79% to 15.05%. Table 2 reports the dataset statistics.

For each user, we rank items by R_{ui}^* and retain the first $\lceil \alpha |\bar{I}_u| \rceil$ items in I_u^α; items absent from every filtered set are removed. All tested values of α leave at least N candidates per user. The baseline uses $\alpha = 40\%$, and the sensitivity analysis considers $\alpha \in \{10\%, 20\%, \ldots, 100\%\}$. The optimization instances may contain slightly fewer users and items than the raw datasets because records that yield no eligible recommendation edge are removed.

Parameters and metrics. The model-effect comparison uses $N \in \{10, 15, 20, 25, 30\}$. Algorithm comparisons fix $N = 10$ and use nominal coverage ratios of 0%, 20%, 40%, 60%, 80%, and 100%. For each ratio, D is the ceiling of the ratio times the number of items incident to at least one candidate edge. The displayed nominal ratio therefore differs from the implemented integer ratio by at most one item. If $D > K_{\max}(\alpha)$, exact methods report the maximum attainable coverage and its optimal predicted utility. Following the aggregate-diversity literature (Adomavicius and Kwon,

2012, 2014; Muter and Aytekin, 2017), we define

$$\mathrm{APS}(x) = \frac{1}{N|U|} \sum_{u \in U} \sum_{i \in I_u^\alpha} R_{ui}^* x_{ui}, \tag{6}$$

$$\mathrm{AD}(x) = \left| \left\{ i \in I^\alpha : \sum_{u \in U_i^\alpha} x_{ui} \geq 1 \right\} \right|. \tag{7}$$

$\mathrm{APS}(x)$ is the average predicted score and serves as an objective-based proxy for recommendation utility; $\mathrm{AD}(x)$ is the number of distinct items in the union of all lists. Figure 3 normalizes coverage by the maximum attainable coverage of the fixed candidate graph, whereas Figure 4 uses $K_{\max}(1)$ for every value of α.

Low-preference ratio and control design. To test whether filtering changes recommendation content rather than merely reducing model size, define an evaluation-only protection set I_u^β for each user and let $L_u(x)$ be user u's recommendation list. The low-preference recommendation ratio is

$$\mathrm{LPR}_\beta(x) = \frac{1}{N|U|} \sum_{u \in U} \sum_{i \in L_u(x)} \mathbf{1}\{i \notin I_u^\beta\}. \tag{8}$$

It is the share of recommendation positions outside the user's top β fraction; smaller values indicate less reliance on low-ranked items. At identical absolute coverage floors D, we compare the unfiltered model ($\alpha = 100\%$) with the baseline filter ($\alpha = 40\%$) and report APS, AD, and LPR_β for $\beta \in \{10\%, 20\%, 40\%\}$. Paired comparisons use only coverage levels satisfying $D \leq K_{\max}(40\%)$. Parameter β is used only for ex post evaluation and never enters the optimization.

Implementations and hardware. Because the flow equivalence implies an integral linear relaxation, Gurobi-LP provides an independent exact objective benchmark. The explicit-network MCF implementation calls the C++ kernel of OR-Tools SimpleMinCostFlow. TPCAR uses a compiled C++ extension for the implicit exchange graph, Johnson potentials, and Dijkstra. BGCR uses Python/Numba just-in-time compilation. Gurobi runs with one thread. Numba is warmed up once, and compilation time is excluded from reported runtime. Candidate graphs are preprocessed and cached; runtime begins once the cache is available and includes Top-N initialization and optimization, but excludes score generation, candidate sorting, graph construction, array assembly, and cache writing. Baseline graph sizes range from 4.746×10^6 edges for OP to 2.61×10^8 for SO. Experiments run on Windows 11 with an Intel Core i7-13650HX processor (14 cores, 20 threads) and approximately 24 GB of memory. Runs unfinished after 15 hours are recorded as timeouts, except for the complete TG TPCAR run at $D/|I^\alpha| = 100\%$, which is retained as a full-run record but excluded from comparisons restricted to the 15-hour limit.

5.2. Coverage effect and validation framework

Table 3 compares the unconstrained Top-N rule with the proposed model, solved rapidly by BGCR, across list lengths. Tables 4 and 5 separate exact objective validation, computational scalability, actual coverage, and heuristic quality. Figure 2 shows the utility–coverage trajectories of TPCAR and BGCR. Figures 2–4 display OP, Yelp, VG, and TG so that every panel contains a complete trajectory; SO boundary and runtime results appear in the tables.

Across all datasets and list lengths, the proposed model substantially increases aggregate diversity. Relative gains computed from Table 3 range from 29.49% to 757.80%, while the largest relative reduction in average predicted score is approximately 0.63%. These gains depend on concentration in the unconstrained lists. Yelp, TG, and VG have highly concentrated initial recommendations and hence larger coverage gaps; OP begins with higher natural coverage and exhibits smaller marginal gains. Explicit coverage management is therefore most valuable when a small set of frequently recommended items carries a large share of exposure.

Coverage gains and utility losses are asymmetric because diversity counts newly exposed items, whereas average predicted score is computed over all $N|U|$ recommendation positions. This pattern is consistent with prioritizing low-opportunity-cost alternatives, but edge-level utility losses were not recorded separately; the interpretation is structural rather than a causal estimate. Increasing N generally raises natural coverage and creates more positions that can be reallocated, although the displayed losses are not strictly monotone for every dataset.

5.3. Exactness and computational efficiency

Table 4 compares exact methods under identical candidate graphs and coverage targets. Table 5 then determines whether the heuristic reaches each nominal target.

OP provides the independent exactness benchmark. Both binary Gurobi and Gurobi-LP solve every target. Their unscaled objectives are 242823.522986 throughout the low-coverage regime and 242775.428302 at the highest coverage target. The table scales values by 10^3 and rounds to two decimals, so last-digit display differences do not alter agreement at the original precision. TPCAR takes approximately 0.04 seconds in the low-coverage OP regime, compared with 50–55 seconds for Gurobi-LP, a speedup exceeding three orders of magnitude. The generic mixed-integer formulation does not produce a reportable solution on Yelp or the larger instances, whereas TPCAR and explicit-network MCF return exact solutions for multiple targets. The value of the network representation is therefore operational, not merely notational.

The specialized methods also validate the gap-based analysis. On TG, TPCAR runtime grows from 2.67 seconds in the inactive regime to the order of 10^4 seconds at high coverage, consistent with repeated exact augmentations indexed by $\Delta = D - K^0$. When Δ is small, TPCAR most effectively reuses the unconstrained optimum; when Δ is

Table 3
Unconstrained Top-N versus the proposed model across recommendation-list lengths.

		$N = 10$		$N = 15$		$N = 20$		$N = 25$		$N = 30$	
Dataset	Scheme	Avg. score	AD	Avg. score	AD	Avg. score	AD	Avg. score	AD	Avg. score	AD
OP	Unconstrained	4.95	1,427	4.95	1,509	4.94	1,572	4.94	1,614	4.93	1,652
	Proposed	4.95	2,367	4.95	2,367	4.94	2,367	4.93	2,367	4.93	2,367
VG	Unconstrained	4.84	3,890	4.83	4,185	4.83	4,405	4.82	4,605	4.81	4,757
	Proposed	4.84	9,988	4.83	9,995	4.82	9,998	4.82	9,998	4.81	10,000
Yelp	Unconstrained	4.81	900	4.80	992	4.79	1,047	4.78	1,098	4.77	1,147
	Proposed	4.79	6,931	4.78	6,974	4.77	6,992	4.77	7,003	4.76	7,008
SO	Unconstrained	4.97	8,934	4.96	9,514	4.96	9,912	4.96	10,193	4.95	10,467
	Proposed	4.97	17,938	4.96	17,944	4.96	17,949	4.96	17,949	4.95	17,950
TG	Unconstrained	4.38	2,427	4.36	2,907	4.35	3,317	4.33	3,647	4.33	3,988
	Proposed	4.37	11,700	4.35	11,709	4.34	11,709	4.33	11,709	4.32	11,709

Note: The unconstrained scheme independently selects each user's N highest-scoring candidates. The proposed model imposes minimum coverage on the same scores and candidate sets and is solved here by BGCR. Avg. score is average predicted score; AD is the number of distinct items in the union of all lists. Exact feasibility and optimality are examined in Tables 4–5.

large and the full graph fits in memory, optimized MCF can be faster. SO illustrates a different boundary: its canonical $D = 0\%$ MCF baseline completes in 151.40 seconds, while no positive-target MCF run produces a reportable result under the available limits.

BGCR runs in seconds on every dataset, reducing hour-scale exact tasks on Yelp and VG to roughly 6–9 seconds. The trade-off may involve both a lower objective and failure to reach D near the attainable coverage limit. Its objective should therefore be compared with an exact feasible solution only when the reported actual coverage meets the nominal target. In practice, TPCAR is preferred for exactness and small Δ; MCF for large Δ when the explicit network fits in memory; and BGCR for very large graphs or stringent time budgets, with actual coverage and feasibility reported alongside objective value.

5.4. Heuristic speed–quality trade-off

Table 5 reports actual coverage and tests whether BGCR's speed advantage preserves feasibility.

BGCR approaches the target rapidly in most cases but does not cover every candidate item at the 100% target for Yelp, VG, TG, or SO. Near maximum coverage, remaining items may have low candidate degree or conflict strongly with the current assignment. Reaching them can require an intermediate exchange that does not immediately add coverage. TPCAR can represent such compensation through multi-arc shortest paths; BGCR may exhaust all direct feasible replacements.

Figure 2 exhibits a pronounced asymmetry: average predicted score declines slowly while aggregate diversity rises substantially. This pattern is consistent with both algorithms prioritizing low-marginal-loss adjustments and indicates that the candidate graph contains many edges with similar utility but different coverage contributions. The two trajectories nearly coincide for OP and differ only slightly for Yelp and TG. Their separation is clearest at high coverage on VG: TPCAR covers 10,014 items and BGCR 9,988, with average predicted scores of 4.84267 and 4.84194, respectively. The result isolates the value of global exchange chains under severe coverage pressure.

5.5. Coverage and candidate-filtering mechanisms

We next vary the nominal coverage ratio and the filter retention rate. D is obtained by multiplying each nominal ratio by $|I^\alpha|$ and rounding up; if it exceeds $K_{\max}(\alpha)$, the exact method reports the maximum attainable coverage. A smaller α means a stricter preference-eligibility boundary, whereas a larger α broadens the candidate graph.

5.5.1. Sensitivity to $D/|I^\alpha|$

Figure 3 fixes $\alpha = 40\%$ and varies the nominal lower-bound ratio. Actual coverage is normalized by the maximum attainable coverage reached by the exact method at the 100% nominal target.

Average predicted score remains stable until the lower bound becomes active and then declines only slightly. Moving from 0% to 100% reduces average predicted score by approximately 0.02%, 0.49%, 0.04%, and 0.23% for OP, Yelp, VG, and TG, respectively. Actual coverage is constant below natural coverage, increases after this breakpoint, and reaches the maximum attainable level at the final target. The shape is consistent with low-opportunity-cost alternatives being used first and increasingly costly alternatives being required near the attainable coverage limit. This piecewise behavior is also consistent with TPCAR's gap-based design, while Tables 4–5 directly support the runtime–quality trade-off between TPCAR and BGCR.

5.5.2. Sensitivity to the retention rate α

For every α, Figure 4 uses the number of candidate items at $\alpha = 100\%$ as a common nominal target. If this target exceeds $K_{\max}(\alpha)$, the exact method returns $K_{\max}(\alpha)$ and the highest average predicted score at that boundary. Coverage is normalized by $K_{\max}(1)$ to avoid a moving denominator.

Maximum attainable coverage generally increases and then saturates as α expands. OP, VG, and TG approach $K_{\max}(1)$ at relatively small retention rates, indicating that high-ranked candidate edges support most attainable coverage. Yelp requires a broader graph. At $\alpha = 10\%$, Yelp contains 3,541 reachable items but can cover only 3,513 simultaneously, demonstrating why $|I^\alpha|$ and $K_{\max}(\alpha)$ must be distinguished.

Table 4
Optimality and computational efficiency across coverage targets ($N = 10$).

Dataset	Algorithm	Metric	Nominal lower-bound ratio $D/\lvert I^{\alpha}\rvert$					
			0%	20%	40%	60%	80%	100%
OP	Gurobi	Obj./10^3	242.82	242.82	242.82	242.82	242.82	242.78
		Time	79.00 s	114.90 s	150.40 s	185.30 s	221.80 s	255.80 s
	Gurobi-LP	Obj./10^3	242.82	242.82	242.82	242.82	242.82	242.78
		Time	49.60 s	51.90 s	55.10 s	59.20 s	63.40 s	67.20 s
	MCF	Obj./10^3	242.83	242.83	242.83	242.83	242.83	242.78
		Time	0.26 s	4.30 s	8.36 s	12.46 s	16.55 s	20.91 s
	TPCAR	Obj./10^3	242.83	242.83	242.83	242.83	242.83	242.78
		Time	0.04 s	0.04 s	0.04 s	2.75 s	13.90 s	23.60 s
	BGCR	Obj./10^3	242.83	242.83	242.83	242.83	242.83	242.77
		Time	0.58 s	0.58 s	0.58 s	0.58 s	0.60 s	0.61 s
Yelp	Gurobi	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	Gurobi-LP	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	MCF	Obj./10^3	453.22	453.22	453.14	452.75	452.09	Infeas.
		Time	1.93 s	27.08 s	52.06 s	76.93 s	102.01 s	Infeas.
	TPCAR	Obj./10^3	453.22	453.22	452.91	452.08	451.03	451.03
		Time	0.33 s	386.70 s	1,451 s	2,721 s	3,648 s	3,649 s
	BGCR	Obj./10^3	453.22	453.17	452.81	451.90	450.88	450.88
		Time	3.62 s	3.95 s	4.57 s	5.21 s	5.64 s	5.64 s
VG	Gurobi	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	Gurobi-LP	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	MCF	Obj./10^3	1,184.14	1,184.14	1,184.14	1,184.14	1,184.13	1,183.64
		Time	5.55 s	89.19 s	174.95 s	258.53 s	367.23 s	518.32 s
	TPCAR	Obj./10^3	1,184.14	1,184.14	1,184.14	1,184.14	1,184.12	1,183.64
		Time	1.14 s	1.14 s	956.50 s	5,003 s	9,378 s	13,072 s
	BGCR	Obj./10^3	1,184.14	1,184.14	1,184.14	1,184.07	1,183.99	1,183.47
		Time	7.07 s	7.07 s	7.16 s	7.91 s	8.64 s	9.25 s
TG	Gurobi	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	Gurobi-LP	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	MCF	Obj./10^3	849.79	849.79	849.68	849.36	848.80	847.80
		Time	5.23 s	69.71 s	135.83 s	201.60 s	543.57 s	967.44 s
	TPCAR	Obj./10^3	849.79	849.79	849.67	849.33	848.75	847.80
		Time	2.67 s	2.67 s	11,691.77 s	23,120.48 s	37,365.33 s	72,414.42 s
	BGCR	Obj./10^3	849.79	849.79	849.67	849.32	848.72	847.77
		Time	7.61 s	7.61 s	8.38 s	9.17 s	9.95 s	10.70 s
SO	Gurobi	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	Gurobi-LP	Obj./10^3	–	–	–	–	–	–
		Time	–	–	–	–	–	–
	MCF	Obj./10^3	913.57	–	–	–	–	–
		Time	151.40 s	–	–	–	–	–
	TPCAR	Obj./10^3	1,768.05	1,768.05	1,768.05	–	–	–
		Time	14.10 s	14.10 s	14.10 s	> 15 h	> 15 h	> 15 h
	BGCR	Obj./10^3	1,768.05	1,768.05	1,768.05	1,768.05	1,768.02	1,767.53
		Time	14.10 s	14.10 s	14.10 s	14.80 s	15.90 s	16.80 s

Note: Gurobi solves the mixed-integer formulation; Gurobi-LP solves its linear relaxation; and MCF solves the explicit network. *Infeas.* means that the nominal target exceeds maximum attainable coverage. A dash means that the run was attempted but did not produce a reportable result under the available time or memory limits; it does not denote an omitted experiment.

Average predicted score remains nearly constant for OP, VG, and TG as the boundary expands, whereas Yelp declines gradually. Because the experiment tracks a rising $K_{\max}(\alpha)$ whenever the common target is infeasible, different values of α can correspond to different actual coverage. Yelp's decline indicates that a broader candidate set makes additional items simultaneously coverable but requires higher-opportunity-cost assignments. Decreasing α simultaneously strengthens user-level eligibility and compresses the network, but can reduce attainable coverage. Thus $D/|I^{\alpha}|$ controls coverage pressure within a given graph, whereas α controls the preference range available to meet that pressure.

5.5.3. Filtering control at identical coverage

For OP and Yelp, set $D = \lceil qK_{\max}(40\%) \rceil$ for $q \in \{0, 20\%, 40\%, 60\%, 80\%, 100\%\}$ and compare $\alpha = 40\%$ against the unfiltered graph $\alpha = 100\%$. Holding absolute D fixed separates the effect of candidate eligibility from a change in the target itself.

Average predicted score and actual coverage are nearly identical under the two candidate graphs in Figures 5(a)–(b). The low-loss exchanges needed for these targets therefore lie predominantly within each user's top 40% candidates; removing the remaining 60% does not destroy the critical augmenting paths. At the same time, $\mathrm{LPR}_{40\%}$ is identically

Table 5
Solution quality across coverage targets ($N = 10$).

Dataset	Algorithm	Metric	Nominal lower-bound ratio $D/\lvert I^{\alpha}\rvert$					
			0%	20%	40%	60%	80%	100%
OP	Gurobi	Avg. score	4.95	4.95	4.95	4.95	4.95	4.95
		AD	1,094	1,256	1,256	1,452	1,937	2,367
	Gurobi-LP	Avg. score	4.95	4.95	4.95	4.95	4.95	4.95
		AD	1,184	1,406	1,622	1,804	2,022	2,367
	MCF	Avg. score	4.95	4.95	4.95	4.95	4.95	4.95
		AD	780	864	1,189	1,476	1,904	2,367
	TPCAR	Avg. score	4.95	4.95	4.95	4.95	4.95	4.95
		AD	1,334	1,334	1,334	1,451	1,935	2,367
	BGCR	Avg. score	4.95	4.95	4.95	4.95	4.95	4.95
		AD	1,427	1,427	1,427	1,477	1,977	2,367
Yelp	Gurobi	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	Gurobi-LP	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	MCF	Avg. score	4.81	4.81	4.81	4.81	4.80	Infeas.
		AD	352	1,494	2,807	4,211	5,614	Infeas.
	TPCAR	Avg. score	4.81	4.81	4.81	4.80	4.79	4.79
		AD	895	1,875	3,749	5,624	7,008	7,008
	BGCR	Avg. score	4.81	4.81	4.80	4.79	4.78	4.78
		AD	900	1,900	3,750	5,650	6,931	6,931
VG	Gurobi	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	Gurobi-LP	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	MCF	Avg. score	4.84	4.84	4.84	4.84	4.84	4.84
		AD	1,251	2,482	4,033	6,009	8,012	10,014
	TPCAR	Avg. score	4.84	4.84	4.84	4.84	4.84	4.84
		AD	3,623	3,623	4,140	6,210	8,280	10,014
	BGCR	Avg. score	4.84	4.84	4.84	4.84	4.84	4.84
		AD	3,890	3,890	4,140	6,240	8,290	9,988
TG	Gurobi	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	Gurobi-LP	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	MCF	Avg. score	4.38	4.38	4.38	4.38	4.37	4.37
		AD	2,426	2,434	4,684	7,026	9,368	11,709
	TPCAR	Avg. score	4.37	4.37	4.37	4.37	4.37	4.36
		AD	2,426	2,426	4,770	7,154	9,539	11,709
	BGCR	Avg. score	4.37	4.37	4.37	4.37	4.37	4.36
		AD	2,427	2,427	4,777	7,177	9,577	11,700
SO	Gurobi	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	Gurobi-LP	Avg. score	–	–	–	–	–	–
		AD	–	–	–	–	–	–
	MCF	Avg. score	4.98	–	–	–	–	–
		AD	3,298	–	–	–	–	–
	TPCAR	Avg. score	4.96	4.96	4.96	–	–	–
		AD	8,934	8,934	8,934	–	–	–
	BGCR	Avg. score	4.96	4.96	4.96	4.96	4.96	4.96
		AD	8,934	8,934	8,934	11,034	14,734	17,938

Note: Avg. score is the average predicted user–item score, and AD is aggregate diversity (the number of distinct recommended items). *Infeas.* denotes an infeasible fixed-lower-bound MCF target. A dash means that the run was attempted but did not produce a reportable result under the available time or memory limits.

zero under filtering, while the unfiltered optimizer increasingly uses items outside the retained top-40% set as coverage pressure rises. Preference filtering changes the set of admissible exchanges; its apparent protection is not obtained by weakening D.

Figure 6 shows that protection is hierarchical rather than confined to one threshold. Zero recommendations outside the top 40% under filtering follow mechanically from eligibility and are not an independent performance test. More importantly, filtering also reduces low-preference shares at stricter 10% and 20% thresholds, whereas the unfiltered optimizer can increasingly draw on items outside these boundaries. Over the common feasible ranges for OP and Yelp, the high-preference subgraph retains most of the flexibility needed to attain the prescribed coverage while excluding user-level low-ranked positions that a total-score objective can mask. Filtering also reduces OP edges from 1.186×10^7 to 4.746×10^6 and Yelp edges from 8.810×10^7 to 3.524×10^7, approximately 60% in each case.

Taken together, the experiments connect the modeling and computational mechanisms. Preference filtering defines a candidate space with auditable preference support. The

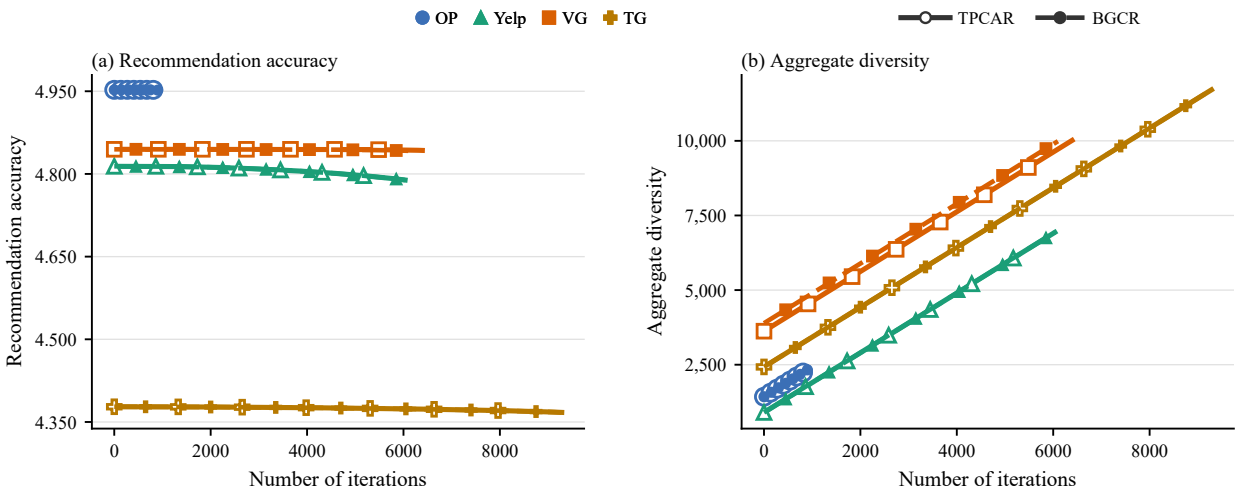

Figure 2: Average predicted score and aggregate-diversity trajectories of TPCAR and BGCR.

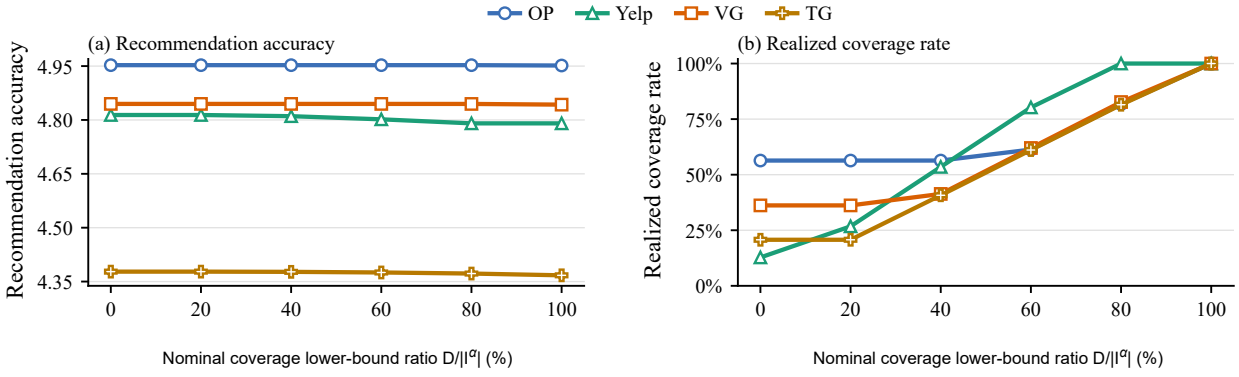

Figure 3: Sensitivity to the nominal coverage lower-bound ratio $D/|I^{\alpha}|$.

single lower-bounded arc concentrates coverage coupling in one network location. TPCAR focuses exact computation on $\max\{D - K^0, 0\}$, whereas BGCR obtains lower runtimes through local batched replacements. The OP benchmark verifies exact objectives independently; Yelp and larger instances show the scale limitations of generic integer programming; and the identical-coverage control demonstrates preference protection without relaxing the common coverage floor.

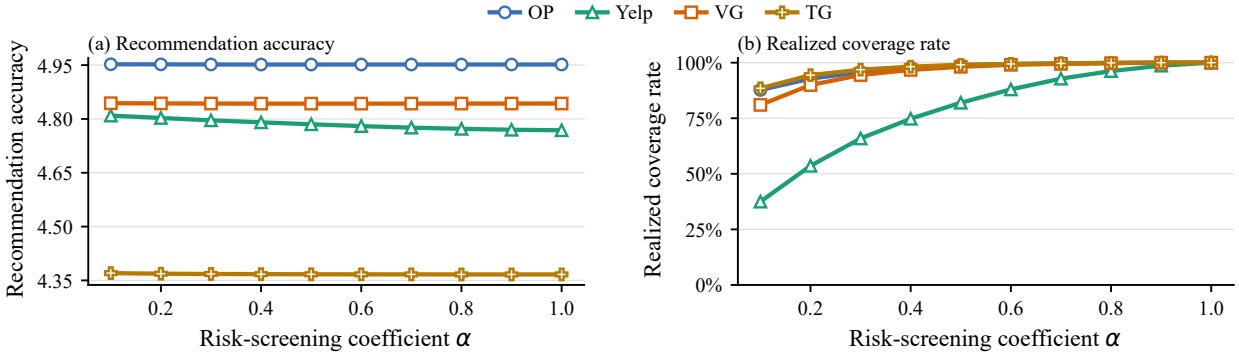

Figure 4: Sensitivity to the candidate-retention rate α.

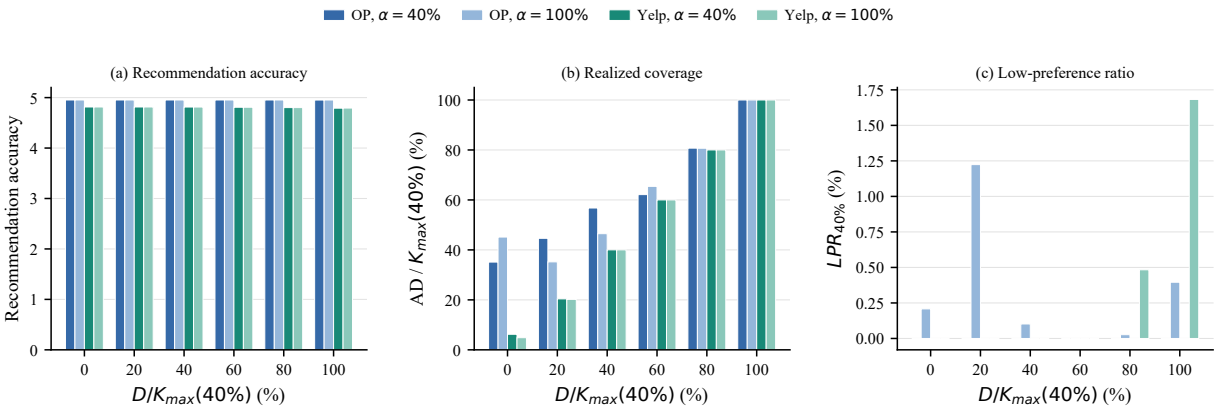

Figure 5: Preference-filtering control at identical coverage lower bounds.

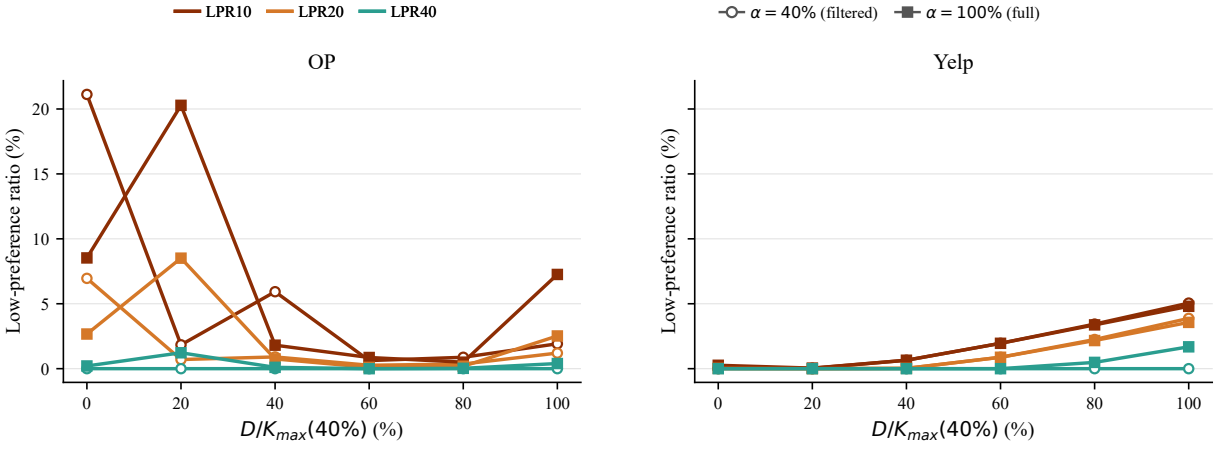

Figure 6: Low-preference recommendation ratios under different protection thresholds.

6. Conclusions

We studied minimum-coverage Top-N recommendation on user-specific high-ranked candidate sets. The coverage floor provides a platform-level mechanism for limiting exposure concentration, while the preference filter confines coverage management to assignments supported by predicted user preferences. Building on earlier total-unimodularity results, we proved that the model is equivalent to a minimum-cost flow problem with one lower-bounded arc. This representation led to two complementary methods for large instances. TPCAR combines a decomposable unconstrained solution with implicit shortest-path augmentations and preserves global optimality. BGCR uses batched local replacements to produce high-quality solutions rapidly and reports whether the requested coverage target is attained.

Experiments on five real-world datasets show that minimum coverage can substantially increase realized aggregate diversity at a small utility cost. On instances with independent exact benchmarks, TPCAR matches the benchmark objective values and is more than three orders of magnitude faster than Gurobi in the low-coverage regime. BGCR reduces several exact computations that require an hour or more to approximately 6–9 seconds. The network structure therefore provides more than an integrality and complexity argument: it supports a practical portfolio of exact and heuristic methods that can be selected according to the realized coverage gap, graph size, required optimality guarantee, and available computing time.

The analysis has two main limitations. First, the formulation adopts a static, offline setting in which predicted scores and candidate sets are treated as fixed. It therefore does not capture the evolution of user feedback, item availability, or prediction error. Second, although TPCAR extends the reach of exact optimization from medium-sized instances to substantially larger candidate graphs, exact computation remains challenging at the largest scales. TPCAR can become time-consuming when the realized coverage gap is large, whereas explicit-network MCF is constrained by memory. Solving very large-scale instances therefore still requires scalable heuristics such as BGCR, which trade global optimality guarantees for computational tractability.

These limitations suggest two directions for future research. The first is to move beyond static assignment

through multiperiod or online formulations that incorporate user feedback, time-varying item states, and score uncertainty. Such models could also extend a single item-coverage floor to categories, sellers, or other supply-side groups, with robust or distributionally robust variants used to protect coverage guarantees against prediction error. The second is to advance exact optimization for very large candidate graphs through parallel shortest-path routines, incremental residual-network updates, graph compression or decomposition, and exact–heuristic hybrids. A central question is whether these techniques can reduce both memory consumption and the runtime sensitivity to large coverage gaps while retaining certifiable solution quality.

CRediT authorship contribution statement

Author A: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. Author B: Methodology, Validation, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No new primary data were collected for this study. The Amazon 5-core review data and Yelp Open Dataset used to construct the experimental instances are publicly available from the sources cited in Section 5.1. The dense predicted-score matrices used as optimization inputs were derived from these third-party data with LETTER. They are not hosted in the GitHub repository because of their size and the terms governing the source data. The repository documents the expected input layout and provides a SHA-256 manifest tool for input verification. Subject to the applicable source terms, the derived inputs are available from the corresponding author on reasonable request.

Code availability

Source code for data preprocessing, the Gurobi and Gurobi-LP benchmarks, explicit minimum-cost flow, TPCAR, BGCR, and the sensitivity analyses is publicly available at https://github.com/qxhuang-123/preference-safe-minimum-coverage. Release v0.1.0 also contains the rebuildable TPCAR C++ extension, processed experiment outputs, environment requirements, and an experiment-to-code crosswalk.

References

Abdollahpouri, H., Burke, R., Mobasher, B., 2017. Controlling popularity bias in learning-to-rank recommendation, in: Proceedings of the 11th ACM Conference on Recommender Systems, pp. 42–46. doi:10.1145/3109859.3109912.

Adomavicius, G., Kwon, Y., 2012. Improving aggregate recommendation diversity using ranking-based techniques. IEEE Transactions on Knowledge and Data Engineering 24, 896–911. doi:10.1109/TKDE.2011.15.

Adomavicius, G., Kwon, Y., 2014. Optimization-based approaches for maximizing aggregate recommendation diversity. INFORMS Journal on Computing 26, 351–369. doi:10.1287/ijoc.2013.0570.

Ahuja, R.K., Magnanti, T.L., Orlin, J.B., 1993. Network Flows: Theory, Algorithms, and Applications. Prentice Hall, Englewood Cliffs, NJ.

Bertsekas, D.P., 1998. Network Optimization: Continuous and Discrete Models. Athena Scientific, Belmont, MA.

Canakoglu, E., Muter, I., Aytekin, T., 2021. Integrating individual and aggregate diversity in top-n recommendation. INFORMS Journal on Computing 33, 300–318. doi:10.1287/ijoc.2019.0952.

Covington, P., Adams, J., Sargin, E., 2016. Deep neural networks for YouTube recommendations, in: Proceedings of the 10th ACM Conference on Recommender Systems, pp. 191–198. doi:10.1145/2959100.2959190.

De Biasio, A., Navarin, N., Jannach, D., 2024. Economic recommender systems: A systematic review. Electronic Commerce Research and Applications 63, 101352. doi:10.1016/j.elerap.2023.101352.

Gao, C., Zheng, Y., Li, N., et al., 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. ACM Transactions on Recommender Systems 1, 1–51. doi:10.1145/3568022.

Goldberg, A.V., Tarjan, R.E., 1990. Finding minimum-cost circulations by successive approximation. Mathematics of Operations Research 15, 430–466. doi:10.1287/moor.15.3.430.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., Chua, T.S., 2017. Neural collaborative filtering, in: Proceedings of the 26th International Conference on World Wide Web, pp. 173–182. doi:10.1145/3038912.3052569.

Koren, Y., 2008. Factorization meets the neighborhood: A multifaceted collaborative filtering model, in: Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 426–434. doi:10.1145/1401890.1401944.

Koren, Y., 2010. Factor in the neighbors: Scalable and accurate collaborative filtering. ACM Transactions on Knowledge Discovery from Data 4, 1–24. doi:10.1145/1644873.1644874.

Koren, Y., Bell, R., Volinsky, C., 2009. Matrix factorization techniques for recommender systems. Computer 42, 30–37. doi:10.1109/MC.2009.263.

Mansoury, M., Abdollahpouri, H., Pechenizkiy, M., Mobasher, B., Burke, R., 2020. Feedback loop and bias amplification in recommender systems, in: Proceedings of the 29th ACM International Conference on Information and Knowledge Management, pp. 2145–2148. doi:10.1145/3340531.3412152.

McAuley, J., Targett, C., Shi, Q., van den Hengel, A., 2015. Image-based recommendations on styles and substitutes, in: Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 43–52. doi:10.1145/2766462.2767755.

Muter, I., Aytekin, T., 2017. Incorporating aggregate diversity in recommender systems using scalable optimization approaches. INFORMS Journal on Computing 29, 405–421. doi:10.1287/ijoc.2016.0741.

Orlin, J.B., 1997. A polynomial-time primal network simplex algorithm for minimum cost flows. Mathematical Programming 78, 109–129. doi:10.1007/BF02614365.

Rendle, S., Freudenthaler, C., Gantner, Z., Schmidt-Thieme, L., 2009. BPR: Bayesian personalized ranking from implicit feedback, in: Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence, pp. 452–461.

Sarwar, B., Karypis, G., Konstan, J., Riedl, J., 2001. Item-based collaborative filtering recommendation algorithms, in: Proceedings of the 10th International Conference on World Wide Web, pp. 285–295. doi:10.1145/371920.372071.

Schrijver, A., 1986. Theory of Linear and Integer Programming. Wiley, Chichester.

Son, J., Kim, H., Kim, S.W., 2025. Rating-aware homogeneous review graphs and user likes/dislikes differentiation for effective recommendations, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 2070–2080. doi:10.1145/3726302.3730069.

Wu, L., Zheng, Z., Qiu, Z., et al., 2024. A survey on large language models for recommendation. World Wide Web 27, 60. doi:10.1007/s11280-024-01291-2.